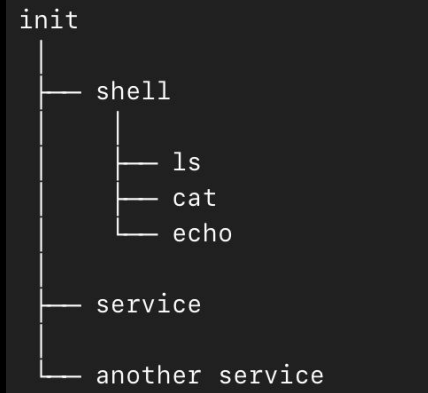


Process APIs

How does a shell work?

Working of Shell

- After bootup, the **init process is first process created**
- The init process spawns a shell like bash
- **All future processes are created by forking from existing processes like init or shell**
- Shell reads user command, forks a child, execs the command executable, waits for it to finish, and reads next command
- Common commands like ls, echo, cat are all readily available executables that are simply exec-ed by the shell



What is Shell?

- A shell is a program that reads commands from the user and asks the OS to execute them.

What is init?

- init is a process. It is the first **user-space process**.

Note: The kernel gets control first, initializes the machine, and eventually starts the first user-space process.

Process APIs: API for Process Management

- What API does OS provide to user programs to manage processes?
 - *How to create, run, terminate processes?*
- API = Application Programming Interface
 - *functions available to write user programs*
- API provided by OS is a set of “system calls”
 - *System call is a function call into OS code that runs at higher CPU privilege level*
 - *Sensitive operations (e.g., access to hardware) are allowed only at a higher privilege level*
- Some “blocking” system calls cause the process to be blocked and context switched out (e.g., `read()` from disk), while others (e.g., `getpid()` to get PID) can return immediately

Process APIs: Should we rewrite programs for each OS?

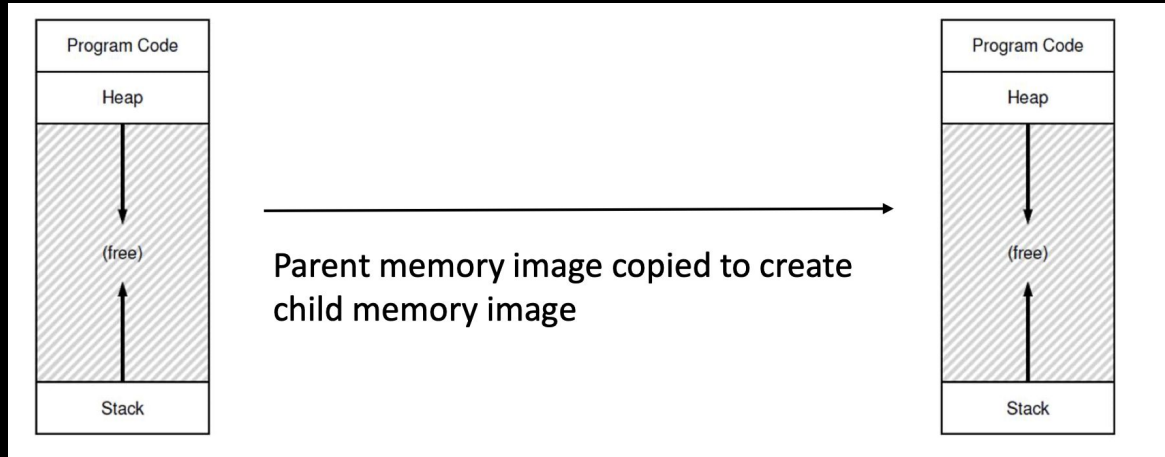
- POSIX API: a standard set of system calls that an OS must implement
 - *Programs written to the POSIX API can run on any POSIX compliant OS*
 - *Most modern OSes are POSIX compliant*
 - *Ensures program portability*
- Program language libraries hide the details of invoking system calls
 - *The printf function in the C library calls the write system call to write to screen*
 - *User programs usually do not need to worry about invoking system calls*

Process APIs: Process related system calls

- `fork()` creates a new child process
 - *All processes are created by forking from a parent*
 - *OS starts init process after boot up, which forks other processes*
 - *The init process is ancestor of all processes, including shell/terminal*
- `exec()` makes a process execute a given executable
- `exit()` terminates a process
- `wait()` causes a parent to block until child terminates
- Many variants of the above system calls exist in language libraries with different arguments

Process API: fork

- Parent process calls “fork” system call to create (spawn) a new process
- New child process created with new PID
- Memory image of parent is copied into that of child
- Parent and child run different copies of same code



Process API: What happens after fork?

- Parent and child resume execution in their copies of the code
- Child starts executing with a **return value of 0** from fork
- Parent resumes executing with a return value equal to child PID
- Parent and child run independently
- Any changes in parent's data after fork does not impact child

```
int ret = fork()
if(ret == 0) {
    print "I am child"
}
else if(ret > 0) {
    print "I am parent"
}
```

Child resumes
here

Parent resumes
here

```
int ret = fork()
if(ret == 0) {
    print "I am child"
}
else if(ret > 0) {
    print "I am parent"
}
```

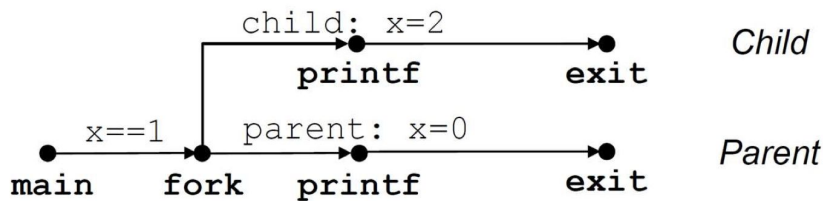
Process API: fork

- What values of x are printed?

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5      int ret = fork();
6      int x = 1;
7
8      if (ret == 0) {
9          printf("I am child\n");
10
11          x = x + 1;
12
13          printf("Child x = %d\n", x);
14      }
15      else if (ret > 0) {
16          printf("I am parent\n");
17
18          x = x - 1;
19
20          printf("Parent x = %d\n", x);
21      }
22      else {
23          printf("fork failed\n");
24      }
25
26      return 0;
27 }
```

Process API: fork

- What values of x are printed?
- Parent and child both start with their own independent copies of variable x in their memory images
- Child increments its copy of x, prints 2
- Parent decrements its copy of x, prints 0



```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5      int ret = fork();
6      int x = 1;
7
8      if (ret == 0) {
9          printf("I am child\n");
10
11         x = x + 1;
12
13         printf("Child x = %d\n", x);
14     }
15     else if (ret > 0) {
16         printf("I am parent\n");
17
18         x = x - 1;
19
20         printf("Parent x = %d\n", x);
21     }
22     else {
23         printf("fork failed\n");
24     }
25
26     return 0;
27 }
```

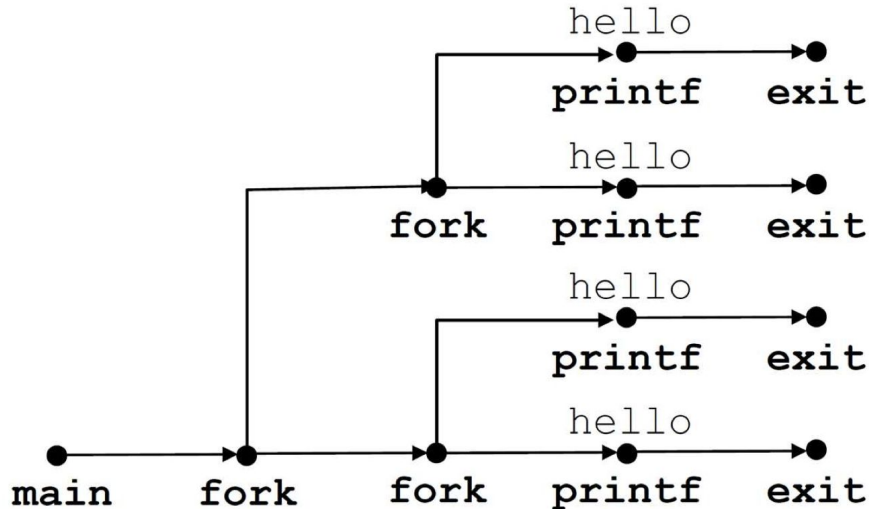
Process API: fork

- Total Processes?
- Number of Child processes?
- How many times hello gets printed?

```
fork()  
fork()  
print hello  
exit
```

Process API: fork

- Total Processes?
- Number of Child processes?
- Total 4 processes (1 parent + 3 child)
- Hello printed 4 times



```
fork()
fork()
printf hello
exit
```

Process API: GATE PYQ 2012

Consider the following code:

The total number of child processes created is:

1. 3
2. 4
3. 7
4. 8

```
fork();  
fork();  
fork();
```

Process API: GATE PYQ 2012

Consider the following code:

The total number of child processes created is:

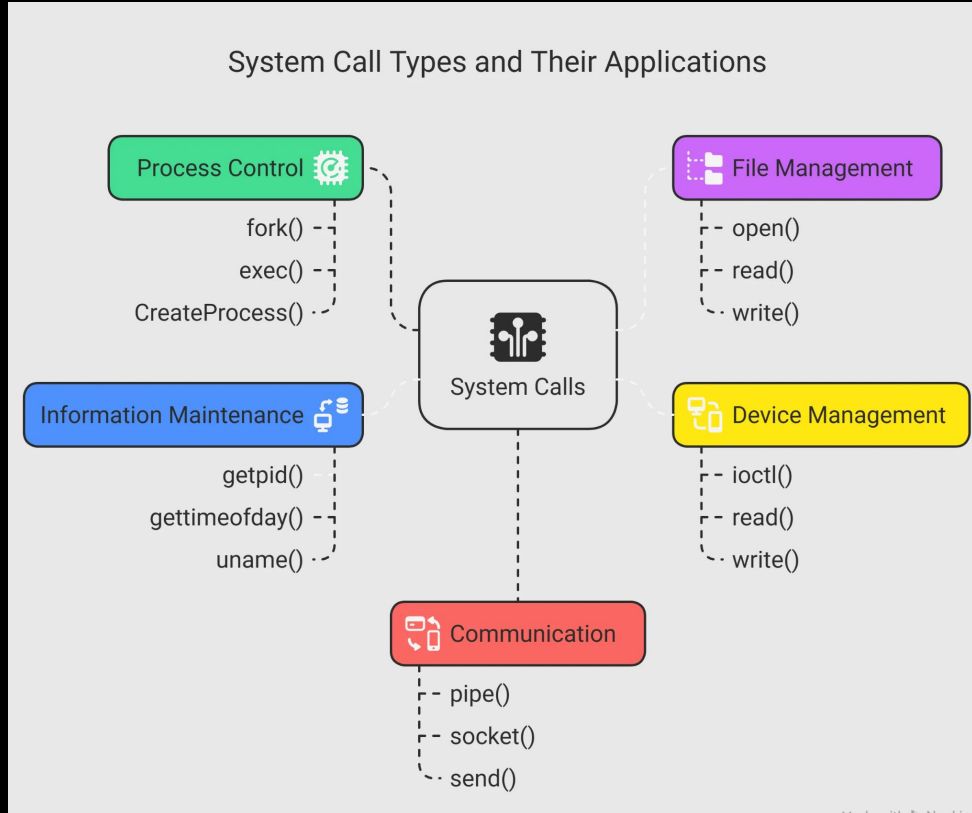
- A. 3
- B. 4
- C. 7
- D. 8

```
fork();  
fork();  
fork();
```

Answer: C. 7

8 total - 1 original parent = 7 children

System Calls other than Process



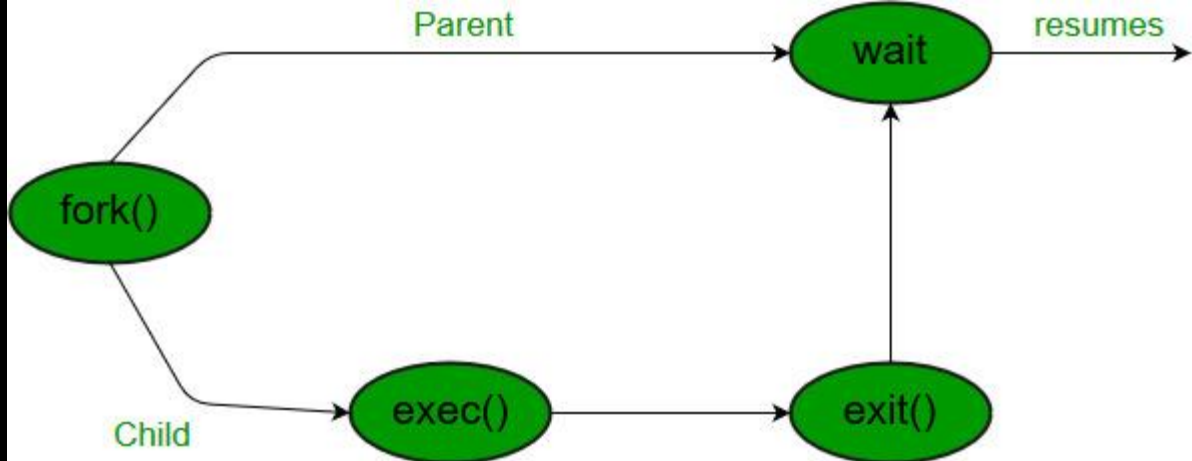
Process API: exec system call

What happens during exec?

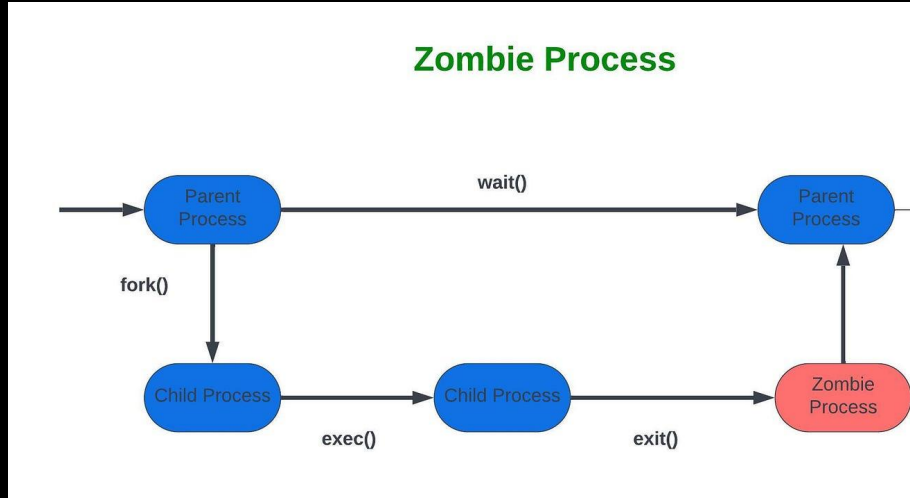
- After fork, parent and child are running same code
 - *Not too useful!*
- A process can run `exec()` to load another executable to its memory image
 - *So, a child can run a different program from parent*
- Variants of `exec()`, e.g., to pass command line arguments to new executable

Process API: wait system call

- Parent calls wait system call to reap (clean up memory of) a **zombie child**
- Wait cleans up memory of one terminated child and returns in parent process
 - If child still running, **wait system call blocks parent until child exits**
 - If child terminated already, **wait reaps child** and returns immediately
- If parent with no child calls wait, it returns immediately without reaping anything



Process: Zombie Process

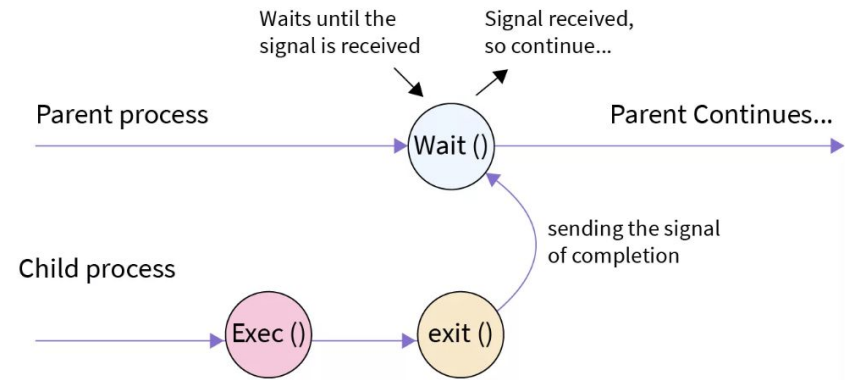


Process table		
Parent PID	→	Parent PCB
...	...	...
Child ID	→	Child PCB
...	...	...

- A zombie process is a dead child process that has finished running, but its entry stays in the system **process table** because the parent process has not yet read its exit status.
- Note: **Process table is a data structure in RAM** to store information about a process.

Process: wait system call

- Wait system call & Zombie process



Before exit() system call

Process table		
Parent PID	→	Parent PCB
...	...	...
Child ID	→	Child PCB
...	...	...

After exit() system call

Process table		
Parent PID	→	Parent PCB
...	...	...
Child pID	→	Child PCB
...	...	...

Zombie process



its indicates that the child process is done with its execution & entered into 'Zombie State'

After exit() system call

Process table		
Parent PID	→	Parent PCB
...	...	...
Child PID	→	Child PCB
...	...	...

Zombie Gone



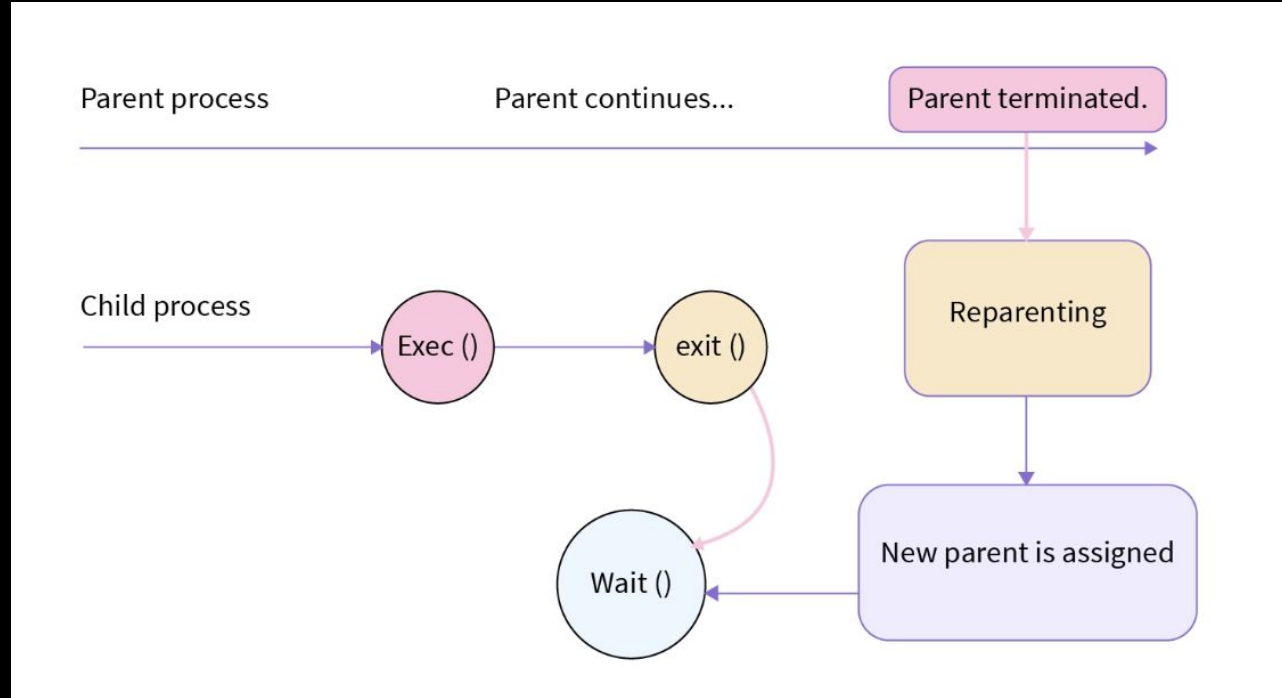
Once the wait() system call is called by the parent, it reads the exit status of the child process and reaps it from the process table

Process API: wait system call

- Wait system call variant waitpid reaps a specific child with a given PID, while regular wait reaps any terminated child
- Wait system call “reaps” one dead child at a time (in any order)
 - *Every fork must be followed by call to wait at some point in parent*
- What if parent has exited while child is still running?
 - *Child will continue to run, becomes orphan*
 - *Orphans adopted by init process, reaped by init when they terminate*
- If parent forks children, but does not bother calling wait for long time, system memory fills up with zombies
 - *Common programming error, exhausts system memory*

Process APIs: Orphan process

- Reparenting
- Parent is dead



Process API: exit system call

- When a process finishes execution, it calls exit system call to terminate
- OS switches the process out and never runs it again
- Exit is automatically called at end of main
- Exiting process cannot clean up its memory, and memory must be freed up by someone else
- Terminated process exists in a zombie state

Process API: Summary

1. fork
2. Exec
3. Wait
4. Zombie process
5. Orphan process
6. init

Function call vs System call

- Function call: application calls another function.
- System call: application requests a service from the OS.

	Normal function call	System call
Called by	Program	Program
Executes in	User code	Kernel code
Privilege	User mode	Kernel mode
CPU privilege transition	No	Yes
Kernel involved	No	Yes
Example	<code>add()</code>	<code>fork()</code> , <code>read()</code> , <code>write()</code>
Why?	Reuse application code	Request OS service